WebBeans Component Guide

2026-10-28

Contents

0. Conventions shared by every component

1. Basic components

- Label
- Button
- TextField
- TextArea
- CheckBox
- ComboBox
- NumberField
- Slider
- RadioGroup
- DatePicker / DateTimePicker
- TimePicker
- ColorPicker
- PasswordField
- TagInput
- ToggleButton
- ButtonGroup
- SplitButton
- Upload
- Form (`org.webbeans.ui.Form`, a helper)
- Link
- Image
- ProgressBar
- Html
- Spacer
- Small display widgets

2. Layout containers

- Panel
- TabbedPane
- SplitPane
- ScrollPane
- Card
- Accordion
- Popover
- Panel busy overlay

3. Flat data: Table, ListBox, Tree

- Table (`org.webbeans.ui.table`)
- ListBox
- Tree + TreeModel
- TreeTable + TreeTableModel (`org.webbeans.ui.table`)
- PropertyGrid
- Downloads — `DownloadService` (`org.webbeans.ui`)

3a. Text, code and output

- CodeEditor
- MarkdownEditor
- DiffView + LineDiff

OutputView

3b. Boards and viewers

Calendar
Kanban
Diagram + Palette
ImageViewer
PdfViewer

4. Explorer views (node-based)

TreeView
OutlineView
ListView
PropertySheet
Enums, labels and value renderers
NodeComboBox

5. Window system and platform widgets

TopComponent
MainWindow, MenuBar, Toolbars, StatusBar
Popup / Menu / MenuItem (programmatic)
QuickSearch (`org.webbeans.quicksearch`, actions module)

6. Dialogs, notifications, progress

DialogDisplayer / NotifyDescriptor
WizardDescriptor
Notifications, status, progress

7. Charts — `Chart`

7a. Graph scenes — `GraphScene<N, E>` (`org.webbeans.graph`)

7b. Maps — `MapView` (`org.webbeans.map`)

8. Scene3D

9. Options and preferences

10. Help

HelpCtx (`org.webbeans.ui`)
@HelpPage (`org.webbeans.help`)
HelpSet, HelpContent, Markdown
HelpViewer — `wb.HelpViewer`
HelpTopComponent, HelpDisplayer, actions, the stand-alone page

11. Platform services worth knowing

12. Quick index

A reference for every component and platform widget **as implemented** (steps 1–12), with a working snippet for each. Read the Developer Guide (`docs/developer-guide.md`) first for the threading rule and the Lookup/registry model; the User's Guide (`docs/user-guide.md`) walks through the same material as a tutorial. All code is Java 8 — anonymous classes where the platform expects a `Listener`, `Function` or `Runnable` (lambdas work too, the examples just stay explicit).

Packages: components in `org.webbeans.ui.components`, the table and tree table in `org.webbeans.ui.table`, `Markdown` and `LineDiff` in `org.webbeans.ui.text`, the base classes, events, `Form`, `EnumLabels`/`Labeled` and `DownloadService` in `org.webbeans.ui`, quick search in `org.webbeans.quicksearch`, windows in `org.webbeans.windows`, actions/dialogs/notifications/progress/options in `org.webbeans.actions`, `org.webbeans.dialogs`, `org.webbeans.notifications`, `org.webbeans.progress`, `org.webbeans.options`, nodes and views in `org.webbeans.nodes` / `org.webbeans.explorer`, help in `org.webbeans.help`, charts in `org.webbeans.charts`, 3D in `org.webbeans.scene3d`.

0. Conventions shared by every component

```
Button save = new Button("Save");                       // convenience constructors set the obvious property
save.set(Button.ENABLED, false);                        // generic typed access: every property is a public static
Property<T>
boolean enabled = save.get(Button.ENABLED);
save.setEnabled(true);                                  // most properties also have fluent setters
Registration r = save.addClickListener(new Listener<ClickEvent>() {
    @Override public void onEvent(ClickEvent e) { doSave(); }
});
r.remove();                                             // keep the Registration to unsubscribe
save.addListener(Component.FOCUS, listener);            // the general form: addListener(EventType, Listener)
save.call("focus");                                     // client-side method invocation (no result); Button has focus()
for it
```

- **Common properties** (`Component`): `VISIBLE`, `ENABLED`, `TOOLTIP`, `STYLE_NAME` (CSS classes, space-separated), `WIDTH`/`HEIGHT` (`Size.px(200)`, `Size.percent(100)`, `Size.fill()`), `ID_FOR_TEST` (`setTestId`, a stable `data-test` attribute used by `HeadlessUI` and Playwright).
- **Common events** (`Component`): `CLICK` (`ClickEvent`: `getButton()`, `getClickCount()`, modifiers), `CONTEXT_MENU`, `FOCUS`, `BLUR`. Events exist on the wire only while a listener is registered.
- **Value components** (`ValueComponent<T>`: `TextField`, `TextArea`, `PasswordField`, `NumberField`, `CheckBox`, `ToggleButton`, `ComboBox`, `RadioGroup`, `ButtonGroup`, `Slider`, `DatePicker`, `DateTimePicker`, `TimePicker`, `ColorPicker`, `TagInput`, `CodeEditor`, `MarkdownEditor`): `getValue()`/`setValue(v)`, `addChangeListener(Listener<ChangeEvent<T>>)` (`e.getValue()`, `e.getOldValue()`, `e.isValid()`), `setValidator(Validator<T>)` (return the error text or `null`; shown inline, sets `ERROR`), `setRequired(true)` (enforced by the platform: "Required", localised), `setReadOnly(true)`, `validate()` (check now, as a change would), `isValid()`. The value is **client-owned**: your `setValue` goes to the browser, the browser's typing is not echoed back — no cursor jumps. A value the server cannot parse (a non-number, a bad date) is rejected: `ERROR` is set and the last valid value is sent back; re-committing the field clears the error. Group fields with a `Form` (§1).
- **`Resource`**: an icon or file inside a module jar — `Resource.of("com/acme/icons/save.svg")`; SVG is preferred (themed with CSS). `Resource.blob(bytes, mime)` for generated images (sent once, cached by hash).
- **Containers** (`Panel`, `SplitPane`, `ScrollPane`, `TabbedPane`, `TopComponent`, `UI.Root`) are `Container`s: `add(c)`, `add(c, constraint)`, `add(c, constraint, index)`, `remove(c)`, `removeAll()`, `getChildren()`. Adding an attached component elsewhere moves it.
- **Keyed items**: components over domain objects (`ComboBox`, `RadioGroup`, `ButtonGroup`, `ListBox`, `Tree`, `Table`, `TreeTable`) send the browser only string keys and rendered labels; `setKeyFunction` (default `toString`) decides the key, and the Java API deals in your objects (`getValue()`, `getSelected()`).
- **Text** is plain and escaped. `Label.HTML`, `Html`, `Cell.html` and help pages accept a sanitised subset (`Html.sanitize`: `<b> <i> <u> <s> <code> <pre> <kbd> <br> <p> <div> <h1-6> <a href> <img src alt> <ul> <ol> <li> <dl> <table> … <blockquote> <hr>`; attributes `href` (http/https/mailto/help/relative), `src` (same origin, https or inline data), `alt`, `title`, `id`, `class`).
- **Threading**: touch components only on the session executor — from an event listener, `ui.access(Runnable)`, `session.access(Runnable)`, or a `UI.Timer`. Everything else (I/O, long work) runs elsewhere and hops back with `access`.

1. Basic components

Label

Props: `TEXT`, `HTML`, `ICON`, `WRAP`.

```
Label status = new Label(Bundle.LBL_Ready());
status.setIcon(Resource.of("com/acme/ok.svg"));
status.setHtml("<b>3</b> " + Bundle.LBL_Warnings());
```

Button

Props: `TEXT`, `ICON`, `KIND` (`Button.Kind.DEFAULT, PRIMARY, DANGER, LINK`), `DEFAULT_BUTTON` (reacts to Enter in its form), `BUSY` (spinner). Methods: `click()`, `focus()`.

```
Button greet = new Button(Bundle.BTN_Greet());
greet.setKind(Button.Kind.PRIMARY).setDefaultButton(true).setTestId("greet");
greet.addClickListener(new Listener<ClickEvent>() {
    @Override public void onEvent(ClickEvent e) { greeting.setText(Bundle.MSG_Hello(name.getValue())); }
});
```

TextField

Props: `VALUE` (client-owned), `PLACEHOLDER`, `MAX_LENGTH`, `PASSWORD`, `PREFIX_ICON`, `CLEAR_BUTTON`, plus `READ_ONLY`, `REQUIRED`, `ERROR`. Events: change (on commit — blur/Enter), `INPUT` (`addInputListener`, while typing, debounced 300 ms), `KEY`.

```
TextField name = new TextField();
name.setPlaceholder("Ada Lovelace").setTestId("name");
name.setRequired(true);
name.setValidator(new Validator<String>() {
    @Override public String validate(String v) { return v == null || v.trim().isEmpty() ? Bundle.ERR_NameRequired() :
null; }
});
name.addChangeListener(new Listener<ChangeEvent<String>>() {
    @Override public void onEvent(ChangeEvent<String> e) { greet.setEnabled(e.isValid()); }
});
name.addInputListener(new Listener<TextField.InputEvent>() {          // live, debounced
    @Override public void onEvent(TextField.InputEvent e) { suggestions.update(e.getValue()); }
});
```

TextArea

As `TextField` plus `ROWS`, `AUTO_GROW`, `MONOSPACE`, `WRAP`; `scrollToLine(n)`.

```
TextArea text = new TextArea();
text.setMonospace(true).setReadOnly(true).setTestId("output");
text.set(TextArea.WRAP, Boolean.FALSE);
text.setValue(content); text.scrollToLine(Integer.MAX_VALUE);
```

CheckBox

Props: `TEXT`, `SELECTED` (= the value, client-owned), `INDETERMINATE`.

```
CheckBox notify = new CheckBox(Bundle.LBL_Notify(), prefs.getBoolean("notify", true));
notify.addChangeListener(new Listener<ChangeEvent<Boolean>>() {
    @Override public void onEvent(ChangeEvent<Boolean> e) { prefs.putBoolean("notify", e.getValue()); }
});
```

ComboBox

Props: `ITEMS`, `PLACEHOLDER`, `FILTERABLE` (type-to-filter), `ALLOW_EMPTY`. Items are keyed (`setKeyFunction`, default `toString`); `setItems(items, labelFn)` / `setItems(items, labelFn, iconFn)`; `getSelectedKey()`.

```
ComboBox<Planet> planet = new ComboBox<Planet>();
planet.setItems(SolarSystem.get().getPlanets(), new Function<Planet, String>() {
    @Override public String apply(Planet p) { return p.getName(); }
});
planet.setAllowEmpty(false).setFilterable(true);
planet.setValue(SolarSystem.get().find("Earth"));
planet.addChangeListener(new Listener<ChangeEvent<Planet>>() {
    @Override public void onEvent(ChangeEvent<Planet> e) { log("planet: " + e.getValue()); }
});
```

NumberField

A numeric input; the value is a `Double` (`null` when empty). Props: `VALUE` (client-owned), `MIN`, `MAX`, `STEP`, `DECIMALS`, `GROUPING`, `SPINNER`, `PREFIX`, `SUFFIX`, `PLACEHOLDER`, and the derived `FORMAT` (decimal and grouping separators of the **session locale**, compiled on the server — the browser formats and parses with it, so `1.234,5` in German and `1,234.5` in English). The server validates the range (`ERR_Min`/`ERR_Max` from the platform bundle); a text that is not a number is rejected with “Not a number” and the last valid value is sent back. `setIntegerOnly()`, `setPattern("#,##0.00")`, `getIntValue()`, `format(double)` (the same formatting for your messages).

```
NumberField price = new NumberField(Double.valueOf(19.99));
price.setDecimals(2).setPrefix("€").setMin(Double.valueOf(0)).setSpinner(true);
NumberField qty = new NumberField(Double.valueOf(1));
qty.setIntegerOnly().setRange(1, 999).setRequired(true);
qty.addChangeListener(new Listener<ChangeEvent<Double>>() {
    @Override public void onEvent(ChangeEvent<Double> e) { total.setText(price.format(qty.getIntValue() *
price.getValue())); }
});
```

Slider

Props: `VALUE` (client-owned `Double`), `MIN`, `MAX`, `STEP`, `TICKS` (a tick every n units), `SHOW_VALUE`, `ORIENTATION`. Events: change on release/keyboard; `INPUT` (`addInputListener`, throttled 50 ms) while dragging — the server value follows the drag only while an input listener is registered.

```
Slider discount = new Slider(0, 50, 10);
discount.setStep(5).setTicks(Double.valueOf(10)).setShowValue(true);
```

RadioGroup

One of several domain objects, as radio buttons. Keyed like `ComboBox` (`setKeyFunction`, default `toString`); `setItems(items)`, `setItems(items, labelFn)`, `setItems(items, labelFn, iconFn)`, `setDescriptions(fn)` (a second line per choice), `ORIENTATION` (`VERTICAL` default, `HORIZONTAL`). An unknown key from the client is rejected (“Unknown choice”).

```
RadioGroup<Shipping> shipping = new RadioGroup<Shipping>(Arrays.asList(Shipping.values()));
shipping.setItems(Arrays.asList(Shipping.values()), new Function<Shipping, String>() {
    @Override public String apply(Shipping s) { return s.label(); }
}).setValue(Shipping.STANDARD).setOrientation(RadioGroup.Orientation.HORIZONTAL);
```

DatePicker / DateTimePicker

The browser’s native pickers; values are `LocalDate` / `LocalDateTime` (ISO-8601 on the wire, no zone). Props: `VALUE` (client-owned), `MIN`, `MAX` (validated on the server: “Must not be before …”), `CLEAR_BUTTON`; `DateTimePicker.SECONDS`. `DatePicker.format(date)` / `DateTimePicker.format(dateTime)` give the session locale’s medium format for messages.

```
DatePicker delivery = new DatePicker(LocalDate.now().plusDays(3));
delivery.setMin(LocalDate.now()).setClearButton(true);
DateTimePicker pickup = new DateTimePicker(LocalDateTime.now().plusHours(2));
```

TimePicker

The browser’s time control; the value is a `LocalTime` (ISO on the wire). Props: `VALUE` (client-owned), `MIN`, `MAX` (validated on the server: “Must not be before 6:00 AM” in the session locale), `STEP_MINUTES` (granularity; `0` shows seconds). `TimePicker.format(time)` for messages.

```
TimePicker launch = new TimePicker(LocalTime.of(9, 30));
launch.setMin(LocalTime.of(6, 0)).setMax(LocalTime.of(22, 0)).setStepMinutes(15);
```

ColorPicker

A swatch that opens a palette popup and, with `ALLOW_CUSTOM`, the browser's colour dialog plus a hex field. The value is a lower-case `#rrggbb` string or `null`. Props: `VALUE` (client-owned), `PALETTE` (defaults to `ColorPicker.DEFAULT_PALETTE`, 22 colours), `ALLOW_CUSTOM`, `SHOW_TEXT`, `CLEARABLE`. A string that is not a colour is rejected ("Invalid value").

```
ColorPicker color = new ColorPicker("#1565c0");
color.setPalette(Arrays.asList("#c62828", "#2e7d32", "#1565c0")).setClearable(true);
```

PasswordField

A `TextField` that is masked from the start, with a reveal button (`REVEAL_BUTTON`) and an optional strength meter (`STRENGTH_METER`, labels Weak/Fair/Good/Strong from the platform bundle). `PasswordField.strength(text)` is the same rule on the server (length and character classes), `getStrength()` the current value's — use it in a `Validator` to insist on `GOOD` or better.

```
PasswordField pw = new PasswordField();
pw.setStrengthMeter(true).setRequired(true).setValidator(new Validator<String>() {
    @Override public String validate(String v) { return PasswordField.strength(v).compareTo(PasswordField.Strength.GOOD) <
0 ? "Too weak" : null; }
});
```

TagInput

A list of short strings as removable chips: Enter or a comma adds, Backspace removes the last one, a `datalist` offers `SUGGESTIONS`. The value is a `List<String>` without duplicates in entry order. Props: `VALUE` (client-owned), `SUGGESTIONS`, `RESTRICT` (only suggestions), `PLACEHOLDER`, `MAX_TAGS` (enforced on the server: "At most n entries"), `LOWER_CASE`. `getTags()`, `setTags`, `addTag`, `removeTag`.

```
TagInput tags = new TagInput(Arrays.asList("rocky", "inner"));
tags.setSuggestions(Arrays.asList("rocky", "inner", "outer", "gas", "ice")).setMaxTags(6);
```

ToggleButton

A button that stays pressed; a `ValueComponent<Boolean>` like `CheckBox` but drawn as a button (`aria-pressed`). Props: `SELECTED` (the value, client-owned), `TEXT`, `ICON`, `SELECTED_ICON` (shown while pressed). `isSelected()`, `setSelected`, `focus()`.

```
ToggleButton orbits = new ToggleButton("Show orbits", true);
orbits.addChangeListener(new Listener<ChangeEvent<Boolean>>() {
    @Override public void onEvent(ChangeEvent<Boolean> e) { scene.setOrbits(orbits.isSelected()); }
});
```

ButtonGroup

A segmented control — one joined button per choice, one pressed at a time: a `RadioGroup` that looks like a toolbar. Same keyed-items API (`setItems(objects[, label[, icon]])`, `setKeyFunction`, `getValue`/`setValue`, `getSelectedKey`); `ButtonGroup.of(MyEnum.class)` lists the constants with their `EnumLabels` labels and icons. Props: `ITEMS`, `ICONS_ONLY` (labels become tooltips), `DESELECTABLE` (clicking the pressed one clears the value).

```
ButtonGroup<Planet.Type> types = ButtonGroup.of(Planet.Type.class);   // "Rocky", "Gas giant", … with their icons
types.setValue(Planet.Type.ROCKY);
```

SplitButton

A main part that fires `click` and an arrow that opens a small menu of `addItem(key, label[, icon])` entries (`addSeparator()`), each firing `SELECT` (`SelectEvent.getKey()`). `setMenuOnly(true)` makes the whole button a menu button. Props: `TEXT`, `ICON`, `KIND` (`Button.Kind`), `ITEMS`, `MENU_ONLY`. For menus of registered *actions* use the actions module's

presenters (`MenuItem`, popups); this one is for a handful of alternatives of one command.

```
SplitButton save = new SplitButton("Save");
save.setKind(Button.Kind.PRIMARY).addItem("saveAs", "Save as…").addItem("saveAll", "Save all");
save.onClick(new Listener<ClickEvent>() { @Override public void onEvent(ClickEvent e) { save(); } });
save.addSelectListener(new Listener<SplitButton.SelectEvent>() {
    @Override public void onEvent(SplitButton.SelectEvent e) { if ("saveAs".equals(e.getKey())) saveAs(); else saveAll();
}
});
```

Upload

A file button with a drop zone. Props: `ACCEPT` (`.csv,text/csv`), `MULTIPLE`, `MAX_SIZE` (bytes; checked in the browser and on the server — the server's own limit is `--max-upload`, default 256 MB), `DROP_ZONE`, `TEXT`, `HINT`, `SHOW_LIST` (the transferred files with progress). The browser POSTs each file to `/wb/upload` (raw body, with the session token); the server stores it in a temporary file and fires `UploadEvent` **on the session executor** — `addUploadListener` is the one listener here that is not a browser event. The temporary file is deleted when the listeners return unless one calls `file.retain()` or `file.moveTo(path)`. Browser-side events: `PROGRESS` (lazy), `FAILED` (`too-large`, `network`, …), `FINISHED` (a whole selection done). `open()` opens the chooser, `clear()` the list.

```
Upload up = new Upload();
up.setAccept(".csv,text/csv").setMaxSize(5L * 1024 * 1024).setHint(Bundle.LBL_DropCsv());
up.addUploadListener(new Listener<Upload.UploadEvent>() {
    @Override public void onEvent(Upload.UploadEvent e) {
        UploadedFile f = e.getFile();                                  // getName, getSize, getContentType, open(),
getBytes(), getPath()
        importCsv(f.open());                                           // or f.moveTo(userDir.resolve(f.getName()))
    }
});
```

Form (`org.webbeans.ui.Form`, a helper)

Groups `ValueComponent`s and tracks their combined validity. `add(fields…)`, `bind(field, consumer)` (where the value goes on `write()`), `enable(button)` (the button's `enabled` follows `isValid()`), `isValid()` (untouched fields count as valid unless required and empty — so Save starts disabled on an empty required form without red errors), `validate()` (checks every field now, showing errors on untouched ones; call it before a submit), `write()`, `reset()`, `setReadOnly(b)`, `firstInvalid()`, `addChangeHandler`. Every `ValueComponent` also has `validate()` and `isValid()` of its own, and `REQUIRED` is enforced by the platform ("Required", localised) for all of them.

```
final Form form = new Form().add(name, email, age).enable(save);
form.bind(name, new Consumer<String>() { @Override public void accept(String v) { contact.setName(v); } });
save.addClickListener(new Listener<ClickEvent>() {
    @Override public void onEvent(ClickEvent e) { if (form.validate()) { form.write(); repository.save(contact);
form.reset(); } }
});
```

Link

Props: `TEXT`, `HREF`, `NEW_TAB`; `setText`, `setHref`. Without `HREF` it emits `CLICK` (a link-styled button). Relative hrefs (`wb/help/files`, `wb/download/<token>`) resolve against the application's base path.

Image

Props: `SOURCE` (`Resource`), `ALT`, `FIT` (`Image.Fit.CONTAIN/COVER/NONE`). `new Image(Resource.of("com/acme/logo.svg"), "Acme")`.

ProgressBar

Props: `VALUE` (0..1), `INDETERMINATE`, `TEXT`. For long operations prefer `ProgressHandle` (§6), which also feeds the status bar.

```
progress.setValue(pct / 100.0).setText(Bundle.MSG_Working(pct));
```

Html

Props: `HTML` (sanitised with `Html.sanitize`). For rich read-only content such as an About box; link clicks open a new tab. (Help pages use the richer `HelpViewer`, §10.)

```
Html about = new Html("<b>Workbench</b> — the WebBeans sample application.<br>Files, a live dashboard …");
```

Spacer

Takes the remaining space in a `Layout.Flow` row (`buttons.add(new Spacer())`), pushing what follows to the end.

Small display widgets

Read-only (or nearly) building blocks for lists, headers and dashboards. All are plain `Component`s with the common properties and `CLICK`.

Component	What it is	Properties / API
`Badge`	a counter or status label (`3`, `NEW`, `Failed`)	`TEXT`, `KIND` (`DEFAULT`, `INFO`, `SUCCESS`, `WARNING`, `DANGER`, `ACCENT`), `PILL` (default) / square, `DOT` (just a coloured dot); `setCount(n)` hides at 0
`Chip`	a token: a filter, a tag, a selected item	`TEXT`, `ICON`, `CLOSABLE` → `addCloseListener`, `SELECTABLE` (click toggles `SELECTED`, client-owned) → `addSelectListener` (`isSelected()` is current), `COLOR` (CSS tint)
`Avatar`	a user picture, or initials on a colour derived from the name	`NAME` (initials: first and last word), `IMAGE`, `SIZE` (px), `COLOR`, `STATUS` (`ONLINE`, `AWAY`, `BUSY`, `OFFLINE` dot), `SQUARE`; `Avatar.initials(name)`
`EmptyState`	the friendly placeholder of an empty list or view	`ICON`, `TITLE`, `TEXT`, `setAction(text, onClick)` (a primary button; its click is the component's `CLICK`)
`Breadcrumb`	a path of crumbs, the last one current	`add(key, label[, icon])`, `setPath(items)`, `truncateTo(key)`, `clear()`, `SEPARATOR`, `MAX_ITEMS` (middle crumbs collapse to "…"); `addSelectListener` (`SelectEvent.getKey()`, `getIndex()`)
`Stepper`	progress through numbered steps (a wizard header, an order's status)	`addStep(title[, description])`, `CURRENT` (`setCurrent`, `next()`, `previous()`, `isFinished()`), `ERROR_STEP`, `ORIENTATION`, `CLICKABLE` (completed steps) + `FREE` (every step) → `addSelectListener` (`getIndex()`)

```
Panel row = new Panel(new Layout.Flow(Layout.Orientation.HORIZONTAL, 8).align(Layout.Align.CENTER));
row.add(new Badge("3", Badge.Kind.ACCENT));
row.add(new Avatar("Ada Lovelace").setStatus(Avatar.Status.ONLINE));
final Chip filter = new Chip("Rocky").setIcon(icon).setSelectable(true);
filter.addSelectListener(new Listener<Event>() { @Override public void onEvent(Event e) {
applyFilter(filter.isSelected()); } });
row.add(filter);
Breadcrumb path = new Breadcrumb().add("home", "Solar system").add("earth", "Earth");
path.addSelectListener(new Listener<Breadcrumb.SelectEvent>() {
    @Override public void onEvent(Breadcrumb.SelectEvent e) { path.truncateTo(e.getKey()); navigate(e.getKey()); }
});
Stepper steps = new Stepper("Choose", "Configure", "Launch").setCurrent(1).setClickable(true);
```

2. Layout containers

Panel

A `Container` with a `Layout`. Props: `LAYOUT`, `PADDING`, `BORDER` (`Panel.Border.NONE/LINE/TITLED` with `TITLE`), `SCROLLABLE`, `SELECTED` (for `Layout.Stack`). `TopComponent` has the same `setLayout`/`setPadding`/`setScrollable`.

Layout	Constraints	Notes
`Layout.Fill`	none	one child fills
`Layout.Border`	`Layout.Border.NORTH/SOUTH/EAST/WEST/CENTER`	the usual choice for a TopComponent
`Layout.Flow(orientation, gap)`	none; `Spacer` pushes	`.align(Layout.Align.START/CENTER/END/STRETCH)`, `.wrap(true)`; toolbars, button rows, vertical stacks
`Layout.Grid(rows, cols, hgap, vgap)`	none (fills row-major) or `Layout.Grid.cell(row, col, rowSpan, colSpan)`	`.fill()` shares the space equally (dashboards); `.columnWidths(Size…)`, `.rowHeights(Size…)`
`Layout.Form`	`Layout.Form.row(label)`, `row(label, hint)`, `section(title)`; or `panel.addRow(label, field)`	two-column forms; `.labelWidth(Size.px(120))`
`Layout.Stack`	none; `panel.setSelected(child)`	wizards, card panels, the Options dialog pages, the Help window's contents/page switch

```
Panel form = new Panel(new Layout.Form().labelWidth(Size.px(120)));
form.add(name, Layout.Form.row(Bundle.LBL_Name(), Bundle.HINT_Name()));
form.add(planet, Layout.Form.row(Bundle.LBL_Planet()));
form.add(subscribe, Layout.Form.row(""));

Panel buttons = new Panel(new Layout.Flow(Layout.Orientation.HORIZONTAL, 8));
buttons.add(greet); buttons.add(new Spacer()); buttons.add(counter);

setLayout(new Layout.Border());                       // inside a TopComponent
add(form, Layout.Border.CENTER);
add(buttons, Layout.Border.SOUTH);

Panel grid = new Panel(new Layout.Grid(2, 2, 8, 8).fill());
grid.setPadding(8);
grid.add(cpu); grid.add(requests); grid.add(traffic);  // row-major
```

TabbedPane

`addTab(title, content)`, `addTab(title, icon, content, closable)`. Props: `SELECTED` (index, client-owned). Events: `addSelectionListener`, `addTabCloseListener`. `getSelectedComponent()`, `setSelectedIndex(i)`.

SplitPane

Two panes with a draggable divider: the first child is the left/top pane. Props: `ORIENTATION` (`HORIZONTAL` = side by side), `POSITION` (first pane in px, client-owned — the user drags it), `MIN_FIRST`/`MIN_SECOND` (`setMinSizes`), `COLLAPSIBLE` (double-click the divider collapses the first pane), `COLLAPSED`. `addChangeListener` (debounced; `e.getInt("position", 0)`), `getFirst()`, `getSecond()`. The window system's splitters between modes are not this component; use it inside a window.

```
SplitPane split = new SplitPane(SplitPane.Orientation.HORIZONTAL, tree, details);
split.setPosition(240).setCollapsible(true);
```

ScrollPane

A scrolling viewport around one child with client-owned offsets. Props: `SCROLL_X`/`SCROLL_Y`, `HORIZONTAL`/`VERTICAL` policies (`ScrollPane.Policy.AUTO/ALWAYS/NEVER`). `scrollTo(x, y)`, `scrollToBottom()`, `scrollIntoView(child)`, `addScrollListener` (`x`, `y`, `atBottom`). `Panel.setScrollable(true)` is the lighter choice when the offsets do not matter.

Card

A `Panel` drawn as a card: an optional header (`CARD_TITLE`, `SUBTITLE`, `ICON`, a banner `IMAGE`) over the children, which the panel's `Layout` places (vertical flow with a gap of 8 by default, padding 12). `CLICKABLE` (`onClick(listener)` sets it) turns the whole card into a button (keyboard too), `CARD_SELECTED` draws the selection ring, `ELEVATED` (default) the shadow. For a gallery, put cards in a `Layout.Flow` with `wrap(true)`.

```
Card card = new Card("Mercury").setSubtitle("Rocky").setIcon(planetIcon).setWidth(Size.px(180));
card.add(new Label("0 moons"));
card.onClick(new Listener<ClickEvent>() { @Override public void onEvent(ClickEvent e) { select(card); } });
```

Accordion

Stacked collapsible sections: `addSection(title, content)` or `addSection(new Accordion.Section(title, icon, badge), content)`; one open at a time unless `setMultiple(true)`. Which sections are open is client-owned (`EXPANDED`, a list of indexes): `setExpanded(0, 2)`, `getExpanded()`, `isExpanded(i)`, `addToggleListener`. The first section added starts open.

Popover

A floating panel anchored to another component — a rich tooltip, a small form under a button. It is a `Container`: add it anywhere in the tree (it renders over everything), point it at an anchor and open it; Escape or a click outside closes it (`CLOSE` event, `OPEN` is client-owned) unless `setCloseOnOutsideClick(false)`. Props: `ANCHOR`, `OPEN`, `PLACEMENT` (`BOTTOM, TOP, LEFT, RIGHT`; flips when there is no room), `ARROW`, `PADDING`. `open()`, `close()`, `toggle()`, `isOpen()`.

```
final Popover details = new Popover(moreButton, detailsPanel);
moreButton.addClickListener(new Listener<ClickEvent>() { @Override public void onEvent(ClickEvent e) { details.toggle(); }
});
panel.add(moreButton); panel.add(details);
```

Panel busy overlay

`panel.setBusy(true, "Scanning…")` dims a panel under a spinner and blocks its input (`BUSY`, `BUSY_TEXT`); `setBusy(false)` when the work is done — the counterpart of `TopComponent.makeBusy` for any part of a window, and of `Button.setBusy` for one button.

3. Flat data: Table, ListBox, Tree

Table (`org.webbeans.ui.table`)

A virtualised data grid over domain objects. Declare columns, hand it a model, listen for selection — the client asks for pages (`PAGE_SIZE`, default 100) as the user scrolls, so a 100 000-row model costs what is on screen. Cells are rendered on the server (text, or a `Cell` with icon/HTML/style); the browser never sees `T`.

```
Table<Order> table = new Table<Order>();
table.setKeyFunction(new Function<Order, String>() { @Override public String apply(Order o) { return
String.valueOf(o.getId()); } });
table.addColumn("id", Bundle.COL_Id(), new Function<Order, Object>() { @Override public Object apply(Order o) { return
o.getId(); } })
     .width(70).align(Column.Align.RIGHT);
table.addColumn("customer", Bundle.COL_Customer(), new Function<Order, Object>() { @Override public Object apply(Order o)
{ return o.getCustomer(); } })
     .minWidth(140).editable(new BiConsumer<Order, String>() {
         @Override public void accept(Order o, String text) {
             if (text.trim().isEmpty()) throw new IllegalArgumentException(Bundle.ERR_Customer());   // shown on the cell,
old text kept
             o.setCustomer(text.trim());
         }
     });
table.addColumn("date", Bundle.COL_Date(), new Function<Order, Object>() { @Override public Object apply(Order o) { return
o.getDate(); } }).width(110);
table.addColumn("total", Bundle.COL_Total(), new Function<Order, Object>() { @Override public Object apply(Order o) {
return o.getTotal(); } })
     .width(110).align(Column.Align.RIGHT);
table.addColumn("status", Bundle.COL_Status(), new Function<Order, Object>() {
    @Override public Object apply(Order o) { return Cell.of(o.getStatus().label(), o.getStatus().icon()).style("status-" +
o.getStatus()); }
}).width(120).sortable(false);

ListTableModel<Order> model = new ListTableModel<Order>(orders);
table.setModel(model).setSelectionMode(Table.SelectionMode.MULTI).setShowFilter(true).setTestId("orders");
table.addSelectionListener(new Listener<Table.SelectionEvent>() {
    @Override public void onEvent(Table.SelectionEvent e) { details.show(table.getSelectedRow()); }
});
table.addActivateListener(new Listener<Table.RowEvent>() {                        // double-click / Enter
    @Override public void onEvent(Table.RowEvent e) { open(e.<Order>getRow()); }
});
table.addRowMenuListener(new Listener<Table.RowEvent>() {                         // right-click: the row is selected
first
    @Override public void onEvent(Table.RowEvent e) { Actions.actionsToPopup(rowActions,
Lookups.fixed(table.getSelected().toArray()), e.getX(), e.getY()); }
});
model.update(order);          // the row re-renders in place (keyed); add/remove/set reset the table and the client re-
requests its pages
```

- **Columns** (`Column<T>`): `title`, `width(px)` / `minWidth(px)` (unset: shares the remaining space), `align`, `sortable` (default on), `resizable`, `hidden`, `format(fn)` (value → text; default: numbers and dates in the session locale, `Boolean` as ✓), `comparator` (default compares `Comparable` values), `editable(consumer)`, `styleName`. `getColumn(id)`, `publishColumns()` after changing a column later (titles on a locale change, say).
- **Cells**: a value function returns a plain value, a `Resource` (icon only) or a `Cell` — `Cell.of(text)`, `Cell.of(text, icon)`, `Cell.html(sanitised)`, `.style(css)`, `.tooltip(t)`. `setRowStyle(fn)` adds classes per row.
- **Models**: `setItems(list)` wraps a `ListTableModel` (`add`, `add(index, row)`, `remove`, `removeAll`, `set`, `clear`, `update(row)`, `replace(old, new)`, `refresh()`, `getItems()`); `LazyTableModel` for data elsewhere — implement `count(Query)` and `fetch(Query, offset, limit)`, both run on a background `RequestProcessor` and the page arrives through `ui.access`; call `refresh()` when the data changed. A `Query` carries the sort column id and direction, the filter text, and a comparator/matcher the table built from the columns (what the in-memory model uses). Your own `TableModel` implements `load(query, offset, limit, callback)` and notifies `Listener.reset()` / `updated(row)`.
- **Sort and filter** are the model's job: `setSort(columnId, asc)` / header clicks (`addSortListener`), `setFilter(text)` / the filter box (`SHOW_FILTER`; the built-in models match the text, case-insensitively, against every visible column's text). Sorting by a column needs `sortable` and comparable values or a `comparator`.
- **Selection** is client-owned keys: `SELECTION_MODE` (`NONE/SINGLE/MULTI`), `getSelected()`, `getSelectedRow()`, `setSelected(rows)`, `setSelectedRow(row)`, `clearSelection()`; `getSelected()` resolves only rows the client has loaded. Keyboard: arrows, Page Up/Down, Home/End, Shift/Ctrl for ranges, Enter activates, F2 edits, Ctrl+A selects all loaded rows.

- **Editing**: double-click or F2 on an `editable` column opens an inline editor; Enter commits (`CellEditEvent` after the column's consumer ran), Escape cancels; a rejected value (an `IllegalArgumentException`) flags the cell with the message for a moment.
- Other props: `ROW_HEIGHT` (28), `SHOW_HEADER`, `STRIPED`, `EMPTY_TEXT` ("No rows", localised). `refresh()` re-requests everything; `scrollTo(row)`, `scrollToIndex(i)`; `getTotal()` is the row count of the current query.

Tabular data that **is** nodes (actions, properties, drag and drop, popup menus for free) is still an `OutlineView` (§4): the Workbench's Files and Server Status windows.

ListBox

A scrolling list of domain objects with single or multiple selection (keys on the wire, like `ComboBox`). `setItems(items[, labelFn[, iconFn]])`, `setDescriptions(fn)`, `setKeyFunction`, `MULTI`, `CHECKBOXES` (a check box per row, implies multi), `VISIBLE_ROWS`, `EMPTY_TEXT`. `getSelected()`, `getSelectedItem()`, `setSelected(items)`, `setSelectedItem(item)`, `scrollTo(item)`. Events: `addSelectionListener` (`e.getSelected()`), `addActivateListener` (double-click/Enter, `e.getItem()`). Click, Shift-click, Ctrl-click, arrows, Space, Ctrl+A.

```
ListBox<City> cities = new ListBox<City>();
cities.setMulti(true).setItems(region.getCities(), new Function<City, String>() { @Override public String apply(City c) {
return c.getName(); } });
cities.addSelectionListener(new Listener<ListBox.SelectionEvent>() {
    @Override public void onEvent(ListBox.SelectionEvent e) { plan(cities.getSelected()); }
});
```

Tree + TreeModel

A plain lazily loaded tree for pickers and simple hierarchies. `TreeModel<T>`: `getChildren(parent)` (`null` = roots; called on the session executor when a node is first expanded), `getLabel`, `getIcon`, `isLeaf` (override when children are expensive), `getKey` (unique in the tree, default `toString`). `Tree.of(childrenFn, labelFn)` for the two-function case. Expansion and selection are client-owned keys: `expand(node)`, `collapse(node)`, `isExpanded`, `getSelected()`, `getSelectedNode()`, `setSelected`, `setSelectedNode`; `refresh()` re-reads roots and expanded nodes, `refresh(node)` one node's children. Events: `addExpandListener` (`getNode()`, `isExpanded()`), `addSelectionListener`, `addActivateListener`. Children already sent are not fetched again (until `refresh`); after a full `init` the expanded nodes' children are re-sent. For nodes with actions, properties, drag and drop and popups use the explorer's `TreeView` (§4).

```
Tree<Path> tree = Tree.of(new Function<Path, List<Path>>() {
    @Override public List<Path> apply(Path dir) { return dir == null ? roots() : list(dir); }
}, new Function<Path, String>() {
    @Override public String apply(Path p) { return p.getFileName().toString(); }
});
tree.addSelectionListener(new Listener<Tree.SelectionEvent>() {
    @Override public void onEvent(Tree.SelectionEvent e) { show(tree.getSelectedNode()); }
});
```

TreeTable + TreeTableModel (`org.webbeans.ui.table`)

A `Table` whose rows form a tree: the first column shows the hierarchy with indentation and expanders, the other columns are ordinary `Column`s — an `OutlineView` for plain objects. `TreeTableModel<T>`: `getRoots()`, `getChildren(parent)` (asked when a node is expanded, so children can be lazy), `isLeaf(node)`, `getKey(node)` (default `toString`); `setTree(roots, childrenFn)` builds one from two functions. The expanded part is **flattened on the server**, so paging, sorting (within siblings) and the filter box (matches are shown with their ancestors, expanded) work as in the flat table and the browser only ever sees rows. Expansion: `expand(node)`, `collapse(node)`, `expandPath(nodes)`, `expandAll()`, `collapseAll()`, `isExpanded`, `getExpandedKeys()`, `addExpandListener` (`getNode()`, `isExpanded()`); keyboard: Right/Left expand, collapse or jump to the parent. Model changes: `structureChanged()` (children added or removed), `nodeChanged(node)` (one row). Everything else — selection, activate, row menus, editable columns, `Cell`s — is the `Table` API.

```
TreeTable<Object> tt = new TreeTable<Object>();
tt.addColumn("Name", new Function<Object, Object>() {
    @Override public Object apply(Object o) { return o instanceof Planet.Type ? Cell.of(label(o), icon(o)) : ((Planet)
o).getName(); }
}).width(200);
tt.addColumn("Moons", new Function<Object, Object>() { @Override public Object apply(Object o) { return o instanceof
Planet ? ((Planet) o).getMoons() : null; } });
tt.setTree(Arrays.<Object>asList(Planet.Type.values()), new Function<Object, List<Object>>() {
    @Override public List<Object> apply(Object o) { return o instanceof Planet.Type ? planetsOf((Planet.Type) o) :
Collections.emptyList(); }
});
tt.setShowFilter(true).expand(Planet.Type.ROCKY);
```

PropertyGrid

A two-column name/value grid over one bean with typed in-place editors — the Properties window for plain objects (the nodes module's `PropertySheet` does the same for `Node`s). Declare properties with a getter and an optional setter; the editor follows the declared type: text (`String`), number (`Integer`, `Long`, `Double`, `BigDecimal`, …), checkbox (`Boolean`), date (`LocalDate`), or a drop-down for enums (automatically, with `EnumLabels` labels and icons) and for `choices(values)` of any type; other types are read-only unless you give a `parser(Function<String, V>)`. Per property: `category` (collapsible groups), `description` (tooltip and the optional description pane, `setShowDescription(true)`), `readOnly()`, `choices`/`choiceLabel`/`choiceIcon`, `renderer(Function<V, Cell>)` for a styled value (icon, html, style, tooltip). `setBean(bean)` reads every value, `refresh()` re-reads them after the bean changed elsewhere. An edit runs the setter on the session thread and fires `CHANGE` (`ChangeEvent.getPropertyId()`, `getOldValue()`, `getNewValue()`, `getBean()`); a setter that throws `IllegalArgumentException` flags the row with its message and keeps the old value.

```
PropertyGrid<Planet> grid = new PropertyGrid<Planet>();
grid.addProperty("name", "Name", String.class, new Function<Planet, String>() { @Override public String apply(Planet p) {
return p.getName(); } },
        new BiConsumer<Planet, String>() {
            @Override public void accept(Planet p, String v) { if (v.trim().isEmpty()) throw new
IllegalArgumentException("Name required"); p.setName(v.trim()); }
        }).category("Identity").description("The planet's name");
grid.addProperty("type", "Type", Planet.Type.class, getType, setType).category("Identity");    // a drop-down of "Rocky",
"Gas giant", …
grid.addProperty("gravity", "Gravity", Double.class, getGravity, setGravity).category("Physics")
    .renderer(new Function<Double, Cell>() { @Override public Cell apply(Double g) { return Cell.of(g + " m/s²").style(g >
15 ? "wb-strong" : null); } });
grid.setBean(planet).setShowDescription(true);
grid.addChangeListener(new Listener<PropertyGrid.ChangeEvent>() {
    @Override public void onEvent(PropertyGrid.ChangeEvent e) { model.planetChanged(grid.getBean(), e.getPropertyId()); }
});
```

Downloads — `DownloadService` (`org.webbeans.ui`)

`DownloadService.offer(bytes, fileName, contentType)` / `offer(Content)` makes the current tab download a file once (a one-time `/wb/download/<token>` URL, five minutes, this session only; streamed by the server, never through the delta channel). `DownloadService.link(producer, text)` returns a `Link` whose URL produces the content on every click (`Supplier<Content>` run on an HTTP thread) for as long as the tab lives; `url(producer)` gives such a URL for your own component. `Content` is `content(bytes, name, mime)`, `content(path, name, mime)` or your own (`getFileName`, `getContentType`, `getLength`, `open()`).

```
DownloadService.offer(csv.getBytes(StandardCharsets.UTF_8), "orders.csv", "text/csv");
panel.add(DownloadService.link(new Supplier<DownloadService.Content>() {
    @Override public DownloadService.Content get() { return DownloadService.content(renderPdf(), "invoice.pdf",
"application/pdf"); }
}, Bundle.LBL_Invoice()));
```

3a. Text, code and output

CodeEditor

A source editor without dependencies: line numbers, syntax highlighting in the browser for `Language.JAVA, JAVASCRIPT, JSON, XML, HTML, CSS, PROPERTIES, MARKDOWN, SQL, SHELL, PYTHON, YAML` (plain otherwise), a gutter with server-kept `Annotation`s (`Kind.ERROR, WARNING, INFO, MARK, BOOKMARK` with a message tooltip), current-line highlight, Tab/Shift+Tab indent and outdent of the selection, Enter auto-indent, soft wrap. A `ValueComponent<String>`: the whole text is the value (`change` on blur, `INPUT` while typing, debounced 300 ms). Props: `VALUE` (client-owned), `LANGUAGE`, `LINE_NUMBERS`, `WRAP`, `TAB_SIZE`, `HIGHLIGHT_CURRENT_LINE`, `PLACEHOLDER`, `ANNOTATIONS`. Methods: `setAnnotations`, `addAnnotation`, `clearAnnotations`, `getAnnotations`, `goToLine(n)`, `setSelection(start, end)`, `insertText`, `focus`, `getLineCount`; events `addInputListener`, `addCursorListener` (`line`, `col`, debounced 200 ms, only sent while listened), `addGutterClickListener` (`line`). Global shortcuts (Ctrl+S …) still reach the application. The text travels as one string — for logs use `OutputView`, for very large files page or split them.

```
CodeEditor code = new CodeEditor(CodeEditor.Language.JAVA);
code.setValue(source);
code.addInputListener(new Listener<TextField.InputEvent>() {
    @Override public void onEvent(TextField.InputEvent e) { code.setAnnotations(compile(e.getValue())); }   //
Annotation(line, Kind.ERROR, message)
});
code.addGutterClickListener(new Listener<Event>() { @Override public void onEvent(Event e) {
toggleBreakpoint(e.getInt("line", 0)); } });
```

MarkdownEditor

A toolbar (heading, bold, italic, code, link, lists, quote; Ctrl+B/I; lists continue on Enter) over a text area with a live preview that the **server** renders through the same `Markdown.toHtml` + `Html.sanitize` the help pages use — so the preview is exactly what an `Html` component or help topic would show. Value: the Markdown source (`change`, `INPUT` debounced 400 ms; `getHtml()` is the rendered HTML). Props: `VALUE`, `PREVIEW` (`SPLIT, OFF, ONLY`; the toolbar button cycles them), `PREVIEW_HTML` (derived), `TOOLBAR`, `PLACEHOLDER`, `ROWS` (fixed height instead of filling). `insertText`, `focus`.

```
MarkdownEditor notes = new MarkdownEditor(task.getNotes());
notes.addChangeListener(new Listener<ChangeEvent<String>>() { @Override public void onEvent(ChangeEvent<String> e) {
task.setNotes(e.getValue()); } });
```

DiffView + LineDiff

Shows the difference between two texts side by side or inline (`Layout.SIDE_BY_SIDE, INLINE`) with line numbers, optional `CodeEditor.Language` highlighting and collapsible unchanged runs (`setContext(n)` keeps n lines around each change, `null` shows all; folds expand on click). The diff is computed on the server by `org.webbeans.ui.text.LineDiff` (Myers' O(ND) algorithm; `diff(a, b)` → `List<Op>` with `type EQUAL/DELETE/INSERT`, `a/b` line indexes and `text`; `unified(ops, context)` prints `@@` hunks) and sent as the edit script, so the client never sees more than it displays. `setTexts(left, right)`, `setLeft`, `setRight`, `setTitles(l, r)`, `setLanguage`, `setWrap`, `setLineNumbers`; `getOps()`, `getAddedCount()`, `getRemovedCount()`, `isIdentical()`, `toUnified(context)`; `addLineClickListener` (`side`, `line`, `op`).

```
DiffView diff = new DiffView(oldText, newText);
diff.setTitles("v1", "v2").setLanguage(CodeEditor.Language.JAVA).setContext(3);
status.setText(diff.getAddedCount() + " added, " + diff.getRemovedCount() + " removed");
```

OutputView

An append-only console (NetBeans' Output window). Lines are appended on the server and streamed in batches (one `append` call per flush, however many `append`s a listener made); each line has a `Style` (`NORMAL, ERROR, WARNING, INFO, SUCCESS, DEBUG, HEADING`; `error(text)`, `warning`, `info`, `success`, `heading` shortcuts), a timestamp (`TIMESTAMPS` shows them), ANSI SGR colours and bold/italic/underline when `ANSI` is on, and URLs become links. A line appended with a **link id** (`append(text, style, "Planet.java:12")`) is a clickable line that fires `LINK` on the server (`LinkEvent.getLink()`, `isUrl()`, `getLine()`) — like `OutputListener`; plain URLs also fire it while listened (otherwise they open in a new tab). The view follows the tail; scrolling up pauses that (`FOLLOW`, client-owned, with a "↓" button to resume; `scrollToEnd()` from the

server). `MAX_LINES` (default 10 000) bounds a ring buffer both on the server (`getLines()`, `getText()`, `getLineCount()`) and in the browser; after a reconnect the whole buffer is re-sent. `clear()`, `setWrap`, `appendAll(lines)`; a text with newlines becomes several lines.

```
final OutputView out = new OutputView().setTimestamps(true);
out.addLinkListener(new Listener<OutputView.LinkEvent>() {
    @Override public void onEvent(OutputView.LinkEvent e) { if (!e.isUrl()) openEditorAt(e.getLink()); }
});
process.onLine(new Consumer<String>() {                    // from a background thread:
    @Override public void accept(final String line) { ui.access(new Runnable() { @Override public void run() {
out.append(line); } }); }
});
```

3b. Boards and viewers

Calendar

A month / week calendar of keyed `Calendar.Entry` objects (`new Entry(key, title, date)` then `.until(end)` for spans, `.at(from, to)` for timed entries, `.color(css)`, `.icon`, `.tooltip`, `.readOnly()`), set with `setEntries(list)`, `addEntry`, `removeEntry`, `clearEntries`. Month and day names follow the session locale; `FIRST_DAY` (`setFirstDayOfWeek`), `WEEK_NUMBERS`, `TOOLBAR` (previous / today / next, month / week), `DAY_START_HOUR`..`DAY_END_HOUR` for the week grid, `DRAGGABLE`. Navigation is the client's (`VIEW`, `DATE`, `SELECTED` are client-owned; `setDate`, `setMonth`, `setView`, `setSelected` from the server) and arrives as `RANGE` (`getFrom()`..`getTo()`, the visible period — load that period's entries there). Interaction: `DAY_CLICK` / `DAY_ACTIVATE` (double-click, `getDate()`), `ENTRY_CLICK` / `ENTRY_ACTIVATE` (`getEntry()`), and `MOVE` when an entry is dragged to another day — the calendar does **not** change the entry itself: the listener decides and typically calls `cal.addEntry(e.getMoved())` (the entry shifted to the drop date). Keyboard: arrows move the selected day, PageUp/Down the period, Enter activates.

```
final Calendar cal = new Calendar().setWeekNumbers(true);
cal.addRangeListener(new Listener<Calendar.RangeEvent>() { @Override public void onEvent(Calendar.RangeEvent e) {
cal.setEntries(load(e.getFrom(), e.getTo())); } });
cal.addMoveListener(new Listener<Calendar.MoveEvent>() { @Override public void onEvent(Calendar.MoveEvent e) {
reschedule(e.getKey(), e.getDate()); cal.addEntry(e.getMoved()); } });
cal.addDayActivateListener(new Listener<Calendar.DayEvent>() { @Override public void onEvent(Calendar.DayEvent e) {
newAppointment(e.getDate()); } });
```

Kanban

Columns (`addColumn(id, title)` → `.color`, `.limit(wip)` (the count turns red beyond it, not enforced), `.addButton(true)` → `ADD` event, `.collapsed`) of keyed `Kanban.Card`s (`new Card(key, title, columnId)` → `.subtitle`, `.tags(...)`, `.icon`, `.color`, `.tooltip`, `.readOnly()`; `setCards`, `addCard`, `updateCard`, `removeCard`, `getCards(columnId)`). The user drags cards between and within columns; unlike the calendar a drop **is applied** on the server (`moveCard` semantics) and then reported as `MOVE` (`getCard()`, `getFromColumn()`, `getColumn()`, `getIndex()`) — a listener that disagrees moves it back. `CARD_CLICK`, `CARD_ACTIVATE`, `CARD_MENU` (right-click, `getX/getY` for a popup), `SELECTED` (client-owned; arrows move it). `DRAGGABLE` switches dragging off.

```
Kanban board = new Kanban();
board.addColumn("todo", "To do").addButton(true); board.addColumn("doing", "Doing").limit(3); board.addColumn("done",
"Done").color("#2e7d32");
for (Task t : tasks) board.addCard(new Kanban.Card(t.getId(), t.getTitle(),
t.getState()).subtitle(t.getOwner()).tags(t.getLabels()));
board.addMoveListener(new Listener<Kanban.MoveEvent>() { @Override public void onEvent(Kanban.MoveEvent e) {
tasks.setState(e.getKey(), e.getColumn()); } });
```

Diagram + Palette

A node-and-edge diagram on an SVG canvas: pan (Shift/middle/Alt-drag, wheel zoom, the +/fit/− buttons, `fitToView()`, `zoomTo`, `scrollTo(key)`), drag nodes (applied on the server, reported as `NODE_MOVE`; `Node.locked(true)` pins one), click/Ctrl-click/rubber-band selection (`SELECTION`, `SELECTED` client-owned, `getSelectedNodes()`), double-click (`NODE_ACTIVATE`, `EDGE_ACTIVATE`), right-click (`NODE_MENU`), Delete (`DELETE` with the selected keys — nothing is removed by the diagram), and with `setConnectable(true)` (or any `addConnectListener`) a port on each node that the user drags onto another node → `CONNECT` (`getFrom()`, `getTo()`; the listener adds the edge). `Node(key, label, x, y)` → `.size`, `.shape(RECT, ROUNDED, ELLIPSE, DIAMOND, CIRCLE)`, `.color`, `.icon`, `.tooltip`, `.style(css)`; `Edge(key, from, to)` → `.label`, `.lineStyle(SOLID, DASHED, DOTTED)`, `.arrow(false)`, `.color`. `addNode`, `addEdge`, `removeNode` (takes its edges), `removeEdge`, `setGraph(nodes, edges)`, `clear()`, `refresh()` after in-place changes, `autoLayout()` (a simple layered layout by edge direction), `GRID` (snap, default 10), `SHOW_GRID`.

A `Palette` is the drag source: `addItem(category, key, label, icon[, description])` in collapsible categories (`SHOW_LABELS`, `SHOW_FILTER`); dragging an item onto a diagram fires the diagram's `DROP` (`getItemKey()`, `getX()`, `getY()` in diagram units) — the listener creates whatever the item stands for. Clicking an item fires the palette's `SELECT`.

```
final Diagram d = new Diagram();
d.addNode("sun", "Sun", 40, 160).shape(Diagram.Shape.CIRCLE).size(70, 70).color("#f9a825").locked(true);
d.addNode("earth", "Earth", 300, 160).icon(planetIcon);
d.addEdge("e1", "sun", "earth").label("1 AU").lineStyle(Diagram.LineStyle.DASHED);
d.addConnectListener(new Listener<Diagram.ConnectEvent>() { @Override public void onEvent(Diagram.ConnectEvent e) {
d.addEdge(newKey(), e.getFrom(), e.getTo()); } });
d.addDropListener(new Listener<Diagram.DropEvent>() { @Override public void onEvent(Diagram.DropEvent e) {
d.addNode(newKey(), e.getItemKey(), e.getX(), e.getY()); } });
d.addDeleteListener(new Listener<Diagram.SelectionEvent>() { @Override public void onEvent(Diagram.SelectionEvent e) { for
(String k : e.getNodeKeys()) d.removeNode(k); } });
Palette palette = new Palette().addItem("Bodies", "moon", "Moon", moonIcon).addItem("Bodies", "comet", "Comet",
cometIcon);
SplitPane split = new SplitPane(SplitPane.Orientation.HORIZONTAL, palette, d);
```

ImageViewer

One image with zoom (wheel, +/−, fit, 1:1, keyboard `+ - 0 1 r`), panning by drag, rotation (`ROTATION`, `rotate()`) and a toolbar with the natural size and zoom. Sources: a module `Resource` (`setSource`), any URL (`setUrl`), or bytes served from the session through the `DownloadService` (`setImage(bytes, contentType)`). `CHECKERBOARD` behind transparent images, `ZOOM` (client-owned; `setZoom(null)`/`fit()` fits).

PdfViewer

The browser’s built-in PDF viewer in an iframe with a file name, a download and an open-in-tab button. `setDocument(bytes, fileName)` or `setDocument(path, fileName)` serve the document from the session (`DownloadService.url`, streamed on request — nothing goes through the delta channel), `setUrl` shows any URL, `setPage(n)` opens at a page. Nothing is parsed on the server; search, print and navigation are the browser’s (headless browsers show nothing — test the download URL instead).

```
PdfViewer pdf = new PdfViewer().setDocument(report.render(), "report.pdf").setPage(2);
```

4. Explorer views (node-based)

All views bind to the `ExplorerManager` of the nearest `ExplorerManager.Provider` ancestor (usually the TopComponent), or to one set with `setExplorerManager`. Data comes from `Node`s; see Developer Guide §8 and User's Guide §7. Common `NodeView` settings: `setSelectionMode(SINGLE/MULTI)`, `setPopupAllowed`, `setDefaultActionAllowed`, `setDragSource`, `setDropTarget`, `setQuickSearchAllowed`.

TreeView

```
TreeView tree = new TreeView();
tree.setRootVisible(false);
tree.setTestId("parts");
tree.expandNode(em.getRootContext());                    // remembered: survives reconnects and root rebuilds
```

Expansion state lives on the server: whatever the user or `expandNode` expanded is expanded again after a reconnect or a root change; `collapseNode` forgets. A node expanded before its parent's children were sent expands as soon as they arrive.

OutlineView

Tree column plus property columns taken from the nodes' `Sheet`s.

```
OutlineView view = new OutlineView(Bundle.COL_Name());
view.setPropertyColumns("size", Bundle.COL_Size(), "modified", Bundle.COL_Modified());
view.setColumnWidth("size", 70).setColumnWidth("modified", 130);
view.setSortable(true).setRootVisible(true);
```

Cells of `PropertySupport.ReadWrite` properties are editable in place; the registered `PropertyEditor` for the value type decides the editor.

ListView

Shows the children of the explored context (a "folder view"): `setShowParentNode`, `setTraversalAllowed`, `setIconSize(px)`.

PropertySheet

Shows the property sets of the selected nodes; multi-selection shows the common properties. The platform's Properties window follows the global selection; embed your own with `setNodes(node)` or `setExplorerManager(em)`; `setExpertVisible(true)` shows expert sets.

Enums, labels and value renderers

Every place that shows an enum — `ComboBox`, `RadioGroup`, `ListBox`, `ButtonGroup`, `Table` cells, `PropertyGrid`, and the node property editors in `OutlineView` / `PropertySheet` — goes through `org.webbeans.ui.EnumLabels.label(value)` (and `icon(value)`), which tries, in order:

1. `Labeled.getDisplayName()` / `getIcon()` / `getDescription()` — implement `org.webbeans.ui.Labeled` on your enum (the demo's `Planet.Type` does, returning `@Messages` texts);
2. a function registered with `EnumLabels.register(Type.class, labelFn[, iconFn])` — for enums you cannot change;
3. the key `<SimpleName>.<CONSTANT>` in the `Bundle.properties` of the enum's package (`Type.GAS_GIANT=Gas giant`, localised through `Bundle_de.properties` like every other message);
4. the prettified constant name: `GAS_GIANT` → `Gas giant`.

`EnumLabels.parse(Type.class, text)` accepts a constant name or a label (case-insensitive), so typed input round-trips. Nothing else is needed for the pretty default; the pieces below are for going further.

Node property editors (`org.webbeans.nodes`). `PropertyEditors.forType(enumType)` is an editor whose `getAsText` is the label, whose `fromText` accepts name or label, and whose `getChoices()` are `PropertyEditor.Choice(value, label, icon)` — the choice icon is shown next to the drop-down in both views. `PropertyEditors.registerEnum(Type.class, labelFn, iconFn)` overrides that for one type, `PropertyEditors.enumEditor(...)` builds one to pass to `Property.setPropertyEditor`. Any editor can decorate the displayed value through `getAsCell(value)` (a `Cell` with icon, html, style, tooltip; the

default uses the enum icon), and a single property can override it with `Node.Property.setRenderer(Function<T, Cell>)` — `getValueAsCell()` is what the views send as the cell decoration (`cell` on the wire), while `getValueAsText()` remains the editable text.

```
public enum Type implements Labeled {                          // option 1: the enum describes itself
    ROCKY, GAS_GIANT, ICE_GIANT, DWARF;
    @Override public String getDisplayName() { return this == ROCKY ? Bundle.TYPE_ROCKY() : this == GAS_GIANT ?
Bundle.TYPE_GAS_GIANT() : …; }   // @Messages("TYPE_ROCKY=Rocky") …
    @Override public Resource getIcon() { return Resource.of("org/acme/icons/" + name().toLowerCase() + ".svg"); }
}
EnumLabels.register(java.time.DayOfWeek.class, new Function<DayOfWeek, String>() {   // option 2: a foreign enum
    @Override public String apply(DayOfWeek d) { return d.getDisplayName(TextStyle.FULL, LocaleContext.get()); }
});
PropertySupport.ReadWrite<Type> type = ...;                     // a node property with a custom cell
type.setRenderer(new Function<Type, Cell>() {
    @Override public Cell apply(Type t) { return Cell.of(EnumLabels.label(t), EnumLabels.icon(t)).style(t == Type.DWARF ?
"wb-muted" : null); }
});
```

NodeComboBox

Lists the children of the explored context in a `ComboBox` and drives the selection: `new NodeComboBox().setExplorerManager(em)`.

Wire the explorer into the window's lookup so the Edit menu, Properties window and context actions follow the selection:

```
ExplorerUtils.activateActions(em, map, true);              // copy/cut/paste/delete in the ActionMap
associateLookup(ExplorerUtils.createLookup(em, map));      // selected nodes + their lookups + the map
```

5. Window system and platform widgets

TopComponent

The unit of the window system — Developer Guide §7. Useful members: `setDisplayName`, `setHtmlDisplayName` (e.g. a bold name while modified), `setIcon`, `setToolTipText`, `makeBusy(true)`, `setClosable(false)`, `setModified(true)`, `open()` / `open(mode)` / `close()` / `requestActive()` / `requestVisible()`, `isOpened()` / `isActive()`, `associateLookup(...)`, `getLookup()`, `getHelpCtx()` / `setHelpCtx(ctx)` (what F1 shows; defaults to the `helpId` of the description); hooks `componentOpened/Closed/Showing/Hidden/Activated/Deactivated`, `canClose()` (return `false` and close later after an async question), `writeState/readState` + a `TopComponent.Factory` for multi-instance windows, `localeChanged()`.

Annotations: `@TopComponent.Registration(mode, openAtStartup, position, roles)`, `@TopComponent.Description(preferredID, iconBase, persistenceType, factory, helpId)`, `@TopComponent.OpenActionRegistration(displayName, position)`; `@Mode.Registration(name, side, defaultSize, position)`; `@TopComponentGroup.Registration`; `@LayoutRegistration(role, resource)`.

MainWindow, MenuBar, Toolbars, StatusBar

Generated by the platform from the registry — do not instantiate. Contribute with annotations (actions, modes) and registry entries: status cells `@Registration(path = "StatusBar", name = …, position = …, type = Component.class)` on a static factory method; toolbars `@FolderRegistration(path = "Toolbars/<name>", displayName = …, position = …)`. Customise the window once per tab with a `MainWindowDecorator` service (`@ServiceProvider(service = MainWindowDecorator.class)`).

Popup / Menu / MenuItem (programmatic)

Normally generated. `Actions.actionsToPopup(actions, context, x, y)` builds a popup for a `CONTEXT_MENU` or `Table.ROW_MENU` event; `new MenuItem().setAction(action, context, false)` presents an action (used by `Presenter.Menu` implementations such as the Workbench's Spin check item).

QuickSearch (`org.webbeans.quicksearch`, actions module)

Edit ▸ Quick Search… (Ctrl+I) opens a search box over the main window that lists what the registered `QuickSearchProvider`s find for the typed text, grouped by category; arrows move, Enter runs, Escape closes. The platform ships providers for **actions** (every `Menu/**` entry, by name or initials — "ot" finds "Open Toolbars" — with the menu path and shortcut as detail; run with the global context), **open windows** (brought to the front) and **help topics** (titles and keywords; opens the Help window). Add your own — searching records, files, customers:

```
@Registration(path = QuickSearch.PROVIDERS, type = QuickSearchProvider.class, position = 500)
public final class CustomerProvider implements QuickSearchProvider {
    @Override public String getCategory() { return Bundle.LBL_Customers(); }
    @Override public void search(String query, Response response) {          // session thread, per keystroke (debounced): keep it quick
        for (final Customer c : Customers.get().find(query, 8)) {
            if (!response.add(c.getName(), c.getCity(), customerIcon, new Runnable() { @Override public void run() { open(c); } })) return;
        }
    }
}
```

`QuickSearch.open(ui)` opens it programmatically (one instance per UI, in the root's overlay slot); `QuickSearch.matches(text, query)` is the contains-or-initials test the providers use.

6. Dialogs, notifications, progress

DialogDisplayer / NotifyDescriptor

Dialogs are asynchronous: `notify(descriptor)` returns a `CompletableFuture<Object>` resolved with the chosen option.

```
DialogDisplayer.getDefault().notify(new NotifyDescriptor.Message(html, NotifyDescriptor.INFORMATION_MESSAGE));

final NotifyDescriptor.InputLine in = new NotifyDescriptor.InputLine(Bundle.LBL_FileName(), Bundle.TITLE_NewFile());
DialogDisplayer.getDefault().notify(in).thenAccept(new Consumer<Object>() {
    @Override public void accept(Object r) {
        if (r == NotifyDescriptor.OK_OPTION) create(in.getInputText().trim());
    }
});

DialogDisplayer.getDefault().notify(new NotifyDescriptor.Confirmation(Bundle.MSG_SaveChanges(name), Bundle.TITLE_Close(),
        NotifyDescriptor.YES_NO_CANCEL_OPTION)).thenAccept(…);   // YES_OPTION / NO_OPTION / CANCEL_OPTION / CLOSED_OPTION

DialogDisplayer.getDefault().notify(new NotifyDescriptor.Exception(ioException));

NotifyDescriptor about = new NotifyDescriptor.Message(html, NotifyDescriptor.INFORMATION_MESSAGE);
about.setHelpCtx(new HelpCtx("workbench"));              // a Help button at the left of the button bar; F1 presses it
```

Custom content and buttons with `DialogDescriptor`:

```
DialogDescriptor d = new DialogDescriptor(content, Bundle.TITLE_About(), true, new Object[] {Bundle.CTL_Close()},
Bundle.CTL_Close(),
        DialogDescriptor.BOTTOM_ALIGN, null);                  // or (…, BOTTOM_ALIGN, new HelpCtx("id"), listener)
d.setMessageType(NotifyDescriptor.PLAIN_MESSAGE);
d.setValid(false);                                    // disables the default option until the form validates (wire it to
Form.isValid())
d.setClosingOptions(new Object[] {NotifyDescriptor.OK_OPTION, NotifyDescriptor.CANCEL_OPTION});   // other buttons keep
the dialog open
d.createNotificationLineSupport().setErrorMessage(Bundle.ERR_Name());
DialogDisplayer.getDefault().notify(d).thenAccept(…);
```

Options are `NotifyDescriptor.OK_OPTION` etc., strings (the label is the result) or `Button`s. `DialogDisplayer.getDefault().createDialog(dd)` returns a long-lived non-modal `Dialog` to `show()`/`dispose()`.

WizardDescriptor

Panels extend `WizardDescriptor.AbstractPanel<D>` (`getComponent`, `isValid`, `readSettings(D)`, `storeSettings(D)`, `fireChange()`; `wizard.putProperty(WizardDescriptor.PROP_ERROR_MESSAGE, text)` shows an error line). Run it:

```
WizardDescriptor<Data> w = new WizardDescriptor<Data>(new WizardDescriptor.ArrayIterator<Data>(namePanel, physicsPanel),
data);
DialogDisplayer.getDefault().notify(w).thenAccept(new Consumer<Object>() {
    @Override public void accept(Object option) { if (option == WizardDescriptor.FINISH_OPTION) create(w.getData()); }
});
```

Notifications, status, progress

```
NotificationDisplayer.getDefault().notify(Bundle.MSG_HighLoad(), Bundle.MSG_HighLoadDetails(load),
NotificationDisplayer.Category.WARNING);
NotificationDisplayer.getDefault(session).notify(title, icon, details, onClick, Priority.NORMAL, Category.INFO);   // from
any thread
StatusDisplayer.getDefault().setStatusText(Bundle.MSG_Saved(name));

ProgressHandle h = ProgressHandleFactory.createHandle(Bundle.CTL_Scan(), new Cancellable() {
    @Override public boolean cancel() { cancelled[0] = true; return true; }
}, session);
h.start(sectors); h.progress(Bundle.MSG_Scanning(i + 1), i); h.finish();     // shown in the status bar of every tab
```

`NotificationDisplayer.getDefault()` and `StatusDisplayer.getDefault()` need the current session (a session task); the `getDefault(session)` / `createHandle(…, session)` forms work from a background thread.

7. Charts — `Chart`

```
Chart cpu = new Chart(ChartType.LINE);
cpu.setTitle(Bundle.CHART_Cpu()).setTooltip(TooltipMode.SHARED).setZoom(ZoomMode.X).setTestId("chart-cpu");
cpu.getXAxis().setType(AxisType.TIME).setFormat("HH:mm:ss");
cpu.getYAxis().setMin(0.0).setMax(100.0).setFormat("#0'%'").addBand(Bundle.BAND_Hot(), 90, 100, "#b43b3b");
cpu.getLegend().setVisible(false);
Series s = cpu.getModel().addSeries("cpu", Bundle.SER_Cpu());
s.setWindow(120);                                          // sliding window of the last 120 points
s.addPoint(Point.of(Instant.now(), load));                 // from a UI.Timer: one delta per point

Chart requests = new Chart(ChartType.BAR);
requests.setLabels(true);
requests.getModel().setCategories(Arrays.asList("/api", "/static", "/ws", "/auth"));
requests.getModel().addSeries("req", Bundle.SER_Requests()).setPoints(points);   // Point.of(category,
y).withKey(category)

Chart traffic = new Chart(ChartType.DONUT);
traffic.getModel().addSeries("share", Bundle.SER_Traffic());        // Point.of(label, y).withLabel(label)

cpu.addPointClickListener(new Listener<PointEvent>() {
    @Override public void onEvent(PointEvent e) { content.set(Collections.singleton(e.getSource()), null); }
});
chart.download("png", "cpu.png");                          // straight to the browser's downloads
chart.exportPng(1200, 700).whenComplete(…);                 // bytes on the server
```

Types: `LINE, AREA, BAR, PIE, DONUT, SCATTER, STEP` (per series too: `addSeries(id, name, type)` for mixed charts). Chart: `setSubtitle, setStacked, setHorizontal, setPalette, setAnimation, setSelectionMode, getY2Axis()` (series `setYAxis(1)`), `addLegendToggleListener, addRangeSelectListener, resetZoom, highlight(seriesId, key), setXRange`. Series: `setColor, setLineStyle, setMarker, setSmooth, setStack(group), setVisible`, keyed `updatePoint(key, p)` / `removePoint(key)`. Axis: `setTitle, setType, setMin/Max, setFormat, setGrid, addMarker(id, value, label, color), addBand`. `ChartTopComponent` wraps one chart with a toolbar.

7a. Graph scenes — `GraphScene<N, E>` (`org.webbeans.graph`)

The Visual Library model: your node and edge objects are mapped to widgets that decide how they look. Override `attachNodeWidget`/`attachEdgeWidget` to choose per object, or configure what `addNode`/`addEdge` return.

```
GraphScene<Service, Call> scene = new GraphScene<Service, Call>() {
    @Override protected NodeWidget attachNodeWidget(Service s) {
        NodeWidget w = newNodeWidget().setLabel(s.getName()).setSecondaryLabel(s.getKind()).setLabelEditable(true)
                .setAccent(s.isUp() ? "#2f8a4a" : "#b43b3b").setAttribute("kind", s.getKind());     // data-kind / --wb-g-
kind for your CSS
        if (s.isDatabase()) { w.setShape(Shape.CYLINDER).addPin("in").setSide(Side.TOP); }
        else { w.setIcon(cube).addPin("http").setLabel("HTTP :8080"); }                          // AUTO pins: rows
inside the node
        return w;
    }
    @Override protected EdgeWidget attachEdgeWidget(Call c) {
        return
newEdgeWidget().setRouter(Router.ORTHOGONAL).setTargetShape(AnchorShape.TRIANGLE).setLabel(c.getProtocol());
    }
};
scene.addNode(api).setLocation(470, 150);
scene.addEdge(call); scene.setEdgeSource(call, web); scene.setEdgeTargetPin(call, db, "in");
scene.addNote("note:1", "Fails over to the replica").setLocation(640,
420).attachTo(scene.findWidget(replica)).setTextEditable(true);
scene.setConnectProvider(new ConnectProvider() {                    // drag from a node's handle or a pin
    public boolean isSource(Widget s) { return s.getObject() instanceof Service; }
    public boolean canConnect(Widget s, Widget t) { return EdgeWidget.nodeOf(s) != EdgeWidget.nodeOf(t); }
    public void connect(Widget s, Widget t) { Call c = new Call(...); scene.addEdge(c).setSource(s).setTarget(t); }
});
scene.setPopupProvider(GraphScene.popupFromRegistry("Services/Actions"));   // context actions over Service / Call
scene.addRenameListener(e -> e.accept());                           // F2 / double-click editing
scene.addDeleteListener(e -> { for (Object o : e.getObjects()) remove(o); });
scene.layout(new TreeGraphLayout().setOrientation(TreeGraphLayout.Orientation.LEFT_RIGHT), true);   // animated
new GraphTopComponent(scene);                                       // toolbar + selection in the lookup
```

Widgets: `Widget` (`setVisible/Opacity/StyleName/Tooltip/Foreground/Background/Border/Attribute`), `NodeWidget` (`setLabel`, `setSecondaryLabel`, `setIcon`, `setIconPosition`, `setShape` — `RECTANGLE, ROUNDED, ELLIPSE, DIAMOND, HEXAGON, PARALLELOGRAM, CYLINDER, DOCUMENT, ICON, CUSTOM` + `setCustomSvg("<path d='M0,0 H{w} …'/>")`, `setLocation` (centre), `setPreferredSize` (else measured by the browser), `setMovable`, `setSelectable`, `setLabelEditable`, `setAccent`, `addPin(id)` → `PinWidget` (`setLabel`, `setIcon`, `setSide` — `AUTO` rows or `LEFT/RIGHT/TOP/BOTTOM` sockets)), `EdgeWidget` (`setSource/Target` (node or pin), `setRouter` — `DIRECT, ORTHOGONAL, CURVED, FREE` + `setControlPoints`, `setSourceAnchor/TargetAnchor` — `CENTER, BOUNDARY, CIRCULAR, LEFT, RIGHT, TOP, BOTTOM, SIDE`, `setSourceShape/TargetShape` — `NONE, ARROW, TRIANGLE[_HOLLOW], DIAMOND[_HOLLOW], CIRCLE[_HOLLOW], BAR`, `setLineStyle`, `setStrokeWidth`, `setCornerRadius`, `setLabel` / `addLabel(text, 0..1, above)`), `NoteWidget` (`setText`, `setLocation` (top-left), `setWidth`, `attachTo`, `setTextEditable`, `setFontSize`).

Scene: `setSelectionMode(NONE, SINGLE, MULTI)`, `setGrid`, `setShowGrid`, `setRouter` (default for edges), `setAnimation`, `setMinimap`, `setDropTarget` (palette drops → `DROP`), `getSelectedObjects/Widgets`, `setSelectedObjects`, `getSelectionLookup`, `fitToView`, `zoomTo`, `centerOn`, `layout(GridGraphLayout | CircularGraphLayout | TreeGraphLayout | ForceGraphLayout, animated)`, `exportSvg/exportPng/download`. Events: `SELECT`, `MOVE` (applied), `CONNECT`, `ACTIVATE` (double-click), `HOVER`, `MENU`, `RENAME`, `DELETE` (nothing removed by itself), `DROP`. `GraphTopComponent` adds the toolbar (zoom, fit, arrange with `setGraphLayout`, save SVG/PNG, `Graph/Actions`). The simpler `Diagram` (§3b) remains for a quick picture without an object model.

7b. Maps — `MapView` (`org.webbeans.map`)

A slippy map with server-owned overlays. Tiles come **through the server** (`TileSource.OSM` is proxied and cached under `<datadir>/system/tiles`; `TileSource.directory` serves a local `z/x/y` folder; `TileSource.direct` needs `ServerConfig.imageSources(origin)`; `TileSource.BLANK` draws a graticule).

```
MapView map = new MapView(TileSource.OSM);
map.setView(LatLng.of(-42.88, 147.33), 11);
map.addLayer("sites", "Sites");
map.addMarker("hq", hq).setLabel("HQ").setColor("#b43b3b").setDraggable(true).setLayer("sites");
map.addPolyline("route", path).setWeight(3).setArrow(true);
map.addCircle("range", hq, 2500).setColor("#0f7f86").setFillOpacity(0.1).setInteractive(false);
map.addPolygon("zone", ring).setFillColor("#b43b3b").addHole(hole);
map.addImage("plan", bounds).setImageUrl("/wb/blob/" + blobId).setOpacity(0.6);
map.addGeoJson("stations", geoJsonText).setPointsAsCircles(true).setLabelProperty("name");  // properties.style per
feature
map.addMapClickListener(e -> { if (e.getOverlay() instanceof Marker) map.openPopup(e.getPosition(), "<b>" + label + "
</b>"); });
map.addDragListener(e -> save(((Marker) e.getOverlay()).getPosition()));
map.addViewListener(e -> status(e.getCenter() + " z" + e.getZoom()));          // lazy; e.getBounds()
map.setDrawMode(MapView.DrawMode.POLYGON);                                     // or the MapTopComponent toolbar
map.addDrawListener(e -> map.addPolygon("area" + n++, e.getPoints()));
map.fitOverlays(40);
new MapTopComponent(map);                                                      // toolbar + Map/Actions
```

Overlays: `Marker` (`setIcon`, `setIconSize`, `setIconAnchor`, `setLabel`, `setColor`, `setDraggable`, `setRotation`), `Polyline` (`setArrow`, `getLength`), `Polygon` (`addHole`, `getArea`), `Circle` (metres), `Rectangle`, `ImageOverlay`, `GeoJsonOverlay`, `Popup` (`setHtml` sanitised, `setText`, `setOpen`); all with `setVisible/Opacity/Tooltip/StyleName/Layer/ZIndex/Interactive/Attribute`, vector ones with `setColor/Weight/Dash/FillColor/FillOpacity/Fill`. Map: `setView`, `panTo`, `setZoom`, `fitBounds`, `setZoomRange`, `setMaxBounds`, `setInteractive`, `setZoomControl`, `setScaleBar`, `setLayerControl`, `setAttribution`, `openPopup`, `closePopups`, `getOverlayBounds`. Events: `VIEW`, `CLICK` (`getOverlay`, `getFeature`, `getFeatureId`, `getPosition`), `DOUBLE_CLICK`, `HOVER`, `MENU`, `DRAG`, `DRAW` (`getMode`, `getPoints`, `getBounds`, `getRadius`), `LAYER_TOGGLE`. `Geo` has `distance`, `bearing`, `destination`, `area`, `length`, `project/unproject`.

8. Scene3D

```
Scene3D scene = new Scene3D();
scene.setGrid(20, 20).setShadows(true).setHoverEffect(HoverEffect.HIGHLIGHT).setBackground(0x0b1020);
scene.getCamera().setPosition(7, 5, 9).lookAt(0, 0, -1);
scene.getControls().setDistanceLimits(3, 60).setPolarLimits(0.05, Math.PI / 2 + 0.2);
DirectionalLight sun = new DirectionalLight(0xffffff, 0.9);
sun.setShadowSize(12); sun.setPosition(8, 14, 10).setCastShadow(true);
scene.getRoot().add(new AmbientLight(0xffffff, 0.45), sun);

Mesh floor = new Mesh("floor", Geometry.plane(30, 30), Material.standard(0x232a33).setRoughness(1));
floor.setRotationEuler(-Math.PI / 2, 0, 0).setPosition(0, -4, 0).setReceiveShadow(true).setPickable(false);
Mesh body = new Mesh(part.getName(), Geometry.box(1, 1, 1),
Material.standard(part.getColor()).setMetalness(0.6).setRoughness(0.4));
body.setCastShadow(true).setUserData(part);
Group g = new Group(part.getName()); g.addChild(body); g.addChild(new Label3D(part.getName())).setPosition(0, 0.5, 0);
scene.getRoot().add(floor, g);

scene.addListener(Scene3D.PICK, new Listener<PickEvent>() {
    @Override public void onEvent(PickEvent e) {
        Part p = e.getUserData() instanceof Part ? (Part) e.getUserData() : null;
        model.setSelected(p);
        if (p != null && e.isDoubleClick()) scene.fit(e.getNode());
    }
});
probe.setDraggable(DragConstraint.plane(Axis.Y, 0).snap(0.5));
scene.addListener(Scene3D.DRAG, new Listener<DragEvent>() {
    @Override public void onEvent(DragEvent e) { if (e.getPhase() == DragEvent.Phase.END && !allowed(e.getPosition()))
e.reject(); }
});
rotor.animate(Animation.rotate(Axis.Z, 1.2));                       // client-side, radians per second
scene.setSelection(Collections.singletonList(node));
scene.snapshot(1280, 720).whenComplete(…);                            // PNG bytes on the server
```

Props: `BACKGROUND`, `FOG` (`Fog.linear(rgb, near, far)`), `GRID`/`GRID_SIZE`, `AXES`, `SHADOWS`, `ANTIALIAS`, `PIXEL_RATIO_CAP`, `HOVER_EFFECT` (`NONE`, `CURSOR`, `HIGHLIGHT`, `OUTLINE`), `SELECTION`/`SELECTION_COLOR`, `STATS`. Events: `PICK` (`getNode()`, `getUserData()`, `getPoint()`, `getNormal()`, `getInstance()`, `isDoubleClick()`), `HOVER`, `DRAG` (`START`/`MOVE`/`END`, `getPosition()`, `reject()`), `CAMERA_CHANGE`, `ANIMATION_DONE`, `READY`.

Node types: `Group`, `Mesh`, `InstancedMesh`, `Line` (`Line.circle(r, segments)`), `Points`, `Label3D`, lights (`AmbientLight`, `HemisphereLight`, `DirectionalLight`, `PointLight`, `SpotLight`). Cameras: `PerspectiveCamera` (default), `OrthographicCamera`; `scene.fit(node)`, `flyTo(state, ms)`. Geometry: `box, sphere, plane, cylinder, cone, torus, circle, ring, Geometry.buffer()` for your own meshes (positions/normals/indices; sent once as a blob), `Gltf.load(stream)`. Materials: `Material.standard/basic/phong/lambert/wireframe` (`setColor`, `setEmissive`, `setMetalness`, `setRoughness`, `setOpacity`, `setSide`), `LineMaterial`, `PointsMaterial`; `Texture.image(bytes, mime)` / `Texture.fromResource(path)`. Keep a scene in sync with a collection: `new SceneBinder<T>(group, keyFn, createFn, updateFn)` + `setItems(items)` / `refresh(item)` / `nodeFor(item)`. Expose the scene graph as nodes: `new AbstractNode(new Scene3DChildren(scene.getRoot()))` + `Scene3DChildren.bindSelection(em, scene)`. `Scene3DTopComponent` wraps a scene with the viewport toolbar (fit, reset view, grid, save PNG, and the actions under `Scene3DTopComponent.ACTIONS_FOLDER` = `Scene3D/Actions`).

9. Options and preferences

One category of Tools ▸ Options is an `OptionsPanelController` registered with `@OptionsPanelController.TopLevelRegistration(id, categoryName, iconBase, keywords, position, roles)` (entry `OptionsDialog/<id>`; `id` defaults to the simple class name). The dialog creates a fresh controller per opening, calls `getComponent(master)` once, `update()` before showing, `applyChanges()` on OK/Apply and `cancel()` otherwise; call `changed()` from your listeners so the dialog enables Apply and re-checks `isValid()`; override `getHelpId()` and the dialog's Help button opens that page while your category is selected. Per-user settings live in `WbPreferences.forModule(cls)` (a `java.util.prefs.Preferences`, stored under the user's data directory; the `forModule(session, cls)` form works outside a session task).

```
@OptionsPanelController.TopLevelRegistration(categoryName = "#CTL_DashboardOptions", iconBase =
"org/webbeans/workbench/dashboard/icons/chart.svg", position = 200)
@Messages({"CTL_DashboardOptions=Dashboard", "LBL_Refresh=Refresh every (seconds)", "LBL_Window=History (points)",
"LBL_Notify=Notify on high load"})
public final class DashboardOptions extends OptionsPanelController {
    static final String PREF_REFRESH = "refreshSeconds", PREF_WINDOW = "window", PREF_NOTIFY = "notifyHighLoad";
    private final NumberField refresh = new NumberField(), window = new NumberField();
    private final CheckBox notify = new CheckBox();
    private Preferences prefs;

    @Override public Component getComponent(Lookup master) {
        prefs = WbPreferences.forModule(DashboardOptions.class);
        refresh.setIntegerOnly().setRange(1, 60);                    // the platform validates the range and shows the
message
        window.setIntegerOnly().setRange(10, 1000);
        Listener<ChangeEvent<Double>> l = new Listener<ChangeEvent<Double>>() {
            @Override public void onEvent(ChangeEvent<Double> e) { changed(); }
        };
        refresh.addChangeListener(l); window.addChangeListener(l);
        notify.addChangeListener(new Listener<ChangeEvent<Boolean>>() {
            @Override public void onEvent(ChangeEvent<Boolean> e) { changed(); }
        });
        Panel p = new Panel(new Layout.Form());
        p.addRow(Bundle.LBL_Refresh(), refresh); p.addRow(Bundle.LBL_Window(), window); p.addRow(Bundle.LBL_Notify(),
notify);
        return p;
    }
    @Override public void update() {
        refresh.setValue(Double.valueOf(prefs.getInt(PREF_REFRESH, 1)));
        window.setValue(Double.valueOf(prefs.getInt(PREF_WINDOW, 120)));
        notify.setValue(prefs.getBoolean(PREF_NOTIFY, true));
    }
    @Override public void applyChanges() {
        if (!isValid()) return;
        prefs.putInt(PREF_REFRESH, refresh.getIntValue());
        prefs.putInt(PREF_WINDOW, window.getIntValue());
        prefs.putBoolean(PREF_NOTIFY, Boolean.TRUE.equals(notify.getValue()));
        DashboardTopComponent.settingsChanged();              // tell the open windows
    }
    @Override public String getHelpId() { return "dashboard.options"; }
    @Override public void cancel() { }
    @Override public boolean isValid() { return refresh.isValid() && window.isValid() && refresh.getValue() != null &&
window.getValue() != null; }
    @Override public boolean isChanged() { return prefs.getInt(PREF_REFRESH, 1) != refresh.getIntValue() || …; }
}
```

Open programmatically: `OptionsDisplayer.getDefault().open("DashboardOptions")` (a `CompletableFuture<Boolean>`: `true` when closed with OK); `open()` shows the first category. Tools ▸ Options is registered by the platform (`Menu/Tools`, position 9000) and only appears when at least one category is registered for the user's roles.

A simple on/off preference without a dialog: `Actions.checkbox(Module.class, "spin", true, onToggle, displayName)` gives a check menu item backed by `WbPreferences` (see `ViewerActions.SpinAction`).

10. Help

HelpCtx (`org.webbeans.ui`)

`new HelpCtx("files.editor")`, `HelpCtx.DEFAULT_HELP` (the home page), `getHelpID()`, `isDefault()`, `display()` (opens the Help window of the current UI through the `HelpCtx.Displayer` service; `false` without a help module or for an unknown id). `HelpCtx.Provider` is implemented by `TopComponent`; `NotifyDescriptor.setHelpCtx` and `OptionsPanelController.getHelpId` attach pages to dialogs and option categories.

@HelpPage (`org.webbeans.help`)

```
@HelpPage(id = "files", parent = "workbench", title = "#HLP_Files", resource =
"org/webbeans/workbench/files/help/files.md", position = 100, keywords = "#KW_Files")
@HelpPage(id = "files.editor", parent = "files", title = "#HLP_Editor", resource =
"org/webbeans/workbench/files/help/editor.md", position = 100)
@Messages({"HLP_Files=Files", "KW_Files=explorer, folder, rename", "HLP_Editor=The editor"})
package org.webbeans.workbench.files;
```

On a class or `package-info.java`; repeatable; `roles` filters like everywhere else. `resource` is a Markdown (`.md`) or HTML fragment in the module jar; `resource_<lang>.ext` next to it serves that language. Relative links to other pages' files become links to those pages; relative images are served from the module; `help:<id>` and `#anchor` (heading slugs) work too; external links open a new tab.

HelpSet, HelpContent, Markdown

`HelpSet.current()` (session thread): the role-filtered, localised table of contents — `getRoots()`, `getHome()` (the first top-level topic), `find(id)`, `byResource(path)`, `getAll()`, `search(query, locale)` (title/keyword hits before text hits); a `Topic` has `getId/getTitle/getResource/getKeywords/getParent/getChildren/getPath`. `HelpContent.html(set, topic, locale)` renders a page (load → `Markdown.toHtml` → link resolution → `Html.sanitize`), `HelpContent.text(...)` strips it for search, `HelpContent.load(resource, locale)` reads the localised variant. `Markdown.toHtml(String)` is a small converter (headings with ids, paragraphs, fenced code, inline code, bold/italic/strike, links, images, auto-links, nested lists, block quotes, rules, pipe tables, raw HTML blocks and inline tags).

HelpViewer — `wb.HelpViewer`

A scrollable, styled page. Props: `HTML` (already sanitised). Events: `NAVIGATE` (`addNavigateListener`; `e.getHref()` is `help:<id>[#anchor]` or `#anchor`). `scrollTo(anchor)`.

HelpTopComponent, HelpDisplayer, actions, the stand-alone page

`HelpTopComponent` (`HelpTC`, its own `help` mode on the right): Contents, Back, search, *Open in new tab* over a `Layout.Stack` of the table of contents (`TreeView` over `HelpNode`s) and the page (breadcrumbs + `HelpViewer`); `show(HelpCtx)`, `showContents()`, `getCurrentTopic()`, `getPageHtml()`, `isShowingContents()`. `HelpDisplayer` is the `HelpCtx.Displayer` service. Actions: `Help ▸ Help Contents` (`org.webbeans.help.HelpContentsAction`) and `F1` (`org.webbeans.help.ContextHelpAction`: the active window's `getHelpCtx()`, else the home page). `HelpPageProvider` serves `GET /wb/help/<id>` as a complete document for signed-in users (a `PageProvider` service, §11). The platform ships the page `platform` ("Getting around").

11. Platform services worth knowing

Service	Declared with	Used for
`MainWindowDecorator`	`@ServiceProvider`	customise every new tab's main window (theme, role layout)
`HelpCtx.Displayer`	`@ServiceProvider` (the help module)	`HelpCtx.display()`
`PageProvider` (`getPrefix()`, `render(session, basePath, path, query)`)	`@ServiceProvider`	full HTML pages outside the single-page app, served to signed-in users and rendered on the session executor (`/wb/help/…`)
`DownloadService`	static, per UI	one-time and reusable download URLs (§3)
`SessionDirectory`	the server	live sessions for administrators
`AuthenticationProvider`	`@ServiceProvider`	accounts (properties file, LDAP, OAuth…)
`Services/Session` registrations	`@Registration(path = "Services/Session", type = …)`	one instance per user session, in `UISession.getLookup()`

HTTP endpoints the components use behind the scenes: `/wb/upload` (raw file bodies from `Upload`, limit `--max-upload`), `/wb/download/<token>` (`DownloadService`), `/wb/blob/<sha256>` (`Resource.blob`, 3D geometry), `/wb/res/<path>` (module resources), `/wb/help/<id>`.

12. Quick index

Need	Component
Text, icon, caption	`Label`, `Html`, `Image`, `Link`
Input	`TextField`, `TextArea`, `PasswordField`, `NumberField`, `CheckBox`, `ToggleButton`, `ComboBox`, `RadioGroup`, `ButtonGroup`, `Slider`, `DatePicker`, `DateTimePicker`, `TimePicker`, `ColorPicker`, `TagInput` (+ `Validator`, `Form`)
Text and code	`CodeEditor`, `MarkdownEditor`, `DiffView` (+ `LineDiff`), `OutputView`
Small widgets	`Badge`, `Chip`, `Avatar`, `EmptyState`, `Breadcrumb`, `Stepper`, `SplitButton`
Files in and out	`Upload` (+ `UploadedFile`), `DownloadService`
Trigger	`Button`, actions + shortcuts (`@ActionReference(path = "Shortcuts", name = "D-S")`)
Arrange	`Panel` + `Layout.Fill/Border/Flow/Grid/Form/Stack` (+ `setBusy`), `TabbedPane`, `SplitPane`, `ScrollPane`, `Card`, `Accordion`, `Popover`, `Spacer`, `setScrollable`
Tabular data	`Table` + `ListTableModel/LazyTableModel/Column/Cell`; `TreeTable` + `TreeTableModel`; `PropertyGrid`
Boards and viewers	`Calendar`, `Kanban`, `Diagram` + `Palette`, `ImageViewer`, `PdfViewer`
Lists and trees of plain objects	`ListBox`, `Tree` + `TreeModel`
Node-based data (actions, properties, DnD)	`OutlineView`, `TreeView`, `ListView`, `PropertySheet`, `NodeComboBox` over `Nodes`; enum labels via `Labeled/EnumLabels`, `PropertyEditors.registerEnum`, `Property.setRenderer`
Windows	`TopComponent`, `@Mode.Registration`, `WindowManager`, `@LayoutRegistration`
Ask / tell the user	`DialogDisplayer` + `NotifyDescriptor/DialogDescriptor/WizardDescriptor`, `NotificationDisplayer`, `StatusDisplayer`, `ProgressHandle`
Menus, toolbars, popups	`@ActionID` + `@ActionRegistration` + `@ActionReference`; `Actions.forPath`, `actionsToPopup`, `MenuItem`; `QuickSearchProvider` (Ctrl+I)
Visualise	`Chart`, `Scene3D` (+ `Scene3DTopComponent`, `SceneBinder`)
Settings	`OptionsPanelController`, `OptionsDisplayer`, `WbPreferences`, `Actions.checkbox`
Help	`@HelpPage` + Markdown pages, `helpId` / `setHelpCtx` / `getHelpId`, `HelpCtx.display()`, `/wb/help/<id>`
Per-user state and push	`UISession` (`getLookup`, `publish/subscribe`, `access`), `Services/Session` registrations, `UI.scheduleAtFixedRate`
Tests	`HeadlessUI` (Developer Guide §14, User's Guide §14)
Anything else	your own `Component` + `@Renderer` (Developer Guide §13)